FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

FLAB3

Charles Gabriel Marcelo	Doc. Hadji Tejuco
Name of Student	Name of Professor
October 29, 2026	October 29, 2026
Data Performed	Date Submitted

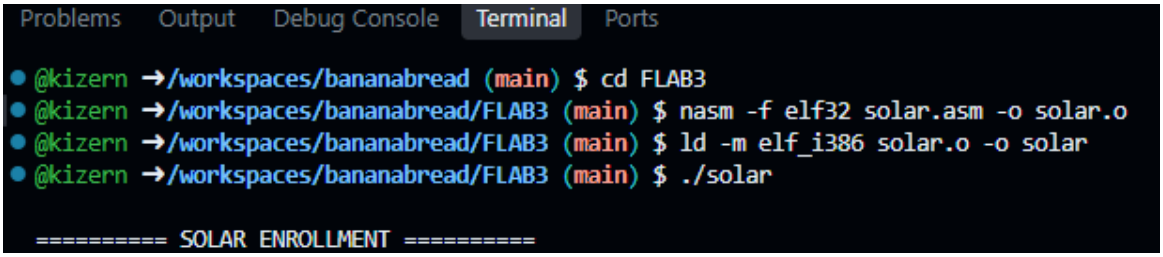
List the steps that you perform to come up with the program.

Paste the source code and explain the functionalities and take a screenshot of the final output.

Program Description

The SOLAR Student Information and Tuition Assessment Program is a console-based application written in NASM 32-bit assembly. The program collects and displays student information, presents the eleven subjects from the screenshot, and accepts zero to thirty units per subject, with zero indicating no enrollment. It calculates tuition at ₱1,000 per unit plus ₱2,500 for each enrolled laboratory subject. Execution begins at _start and proceeds through student information entry, information display, subject unit entry, fee calculation, and assessment display. Macros simplify repeated input and output operations, while procedures handle data collection, validation, number conversion, and calculations. The .data section stores predefined text, .bss reserves memory for inputs and totals, and .text contains executable instructions. Invalid entries trigger another prompt, and the final output shows total units, tuition, laboratory fees, and the total amount due. Records are stored only during execution.

COMPILATION PROCESS:



PROGRAM OUTPUT:

```
========== SOLAR ENROLLMENT ==========
Enrollment status: Enrolled
Student number: 202511693
Name: Charles Gabriel Marcelo
Phone: 67676776767
Zip code: 67676
Classification (or N/A): Regular
College: COMPUTER STUDIES
Program: BSCSSE
Year level: 2nd
Term: 1
School year: 2026-2027
Address: 123 Test. St

---------- STUDENT INFORMATION ----------
Enrollment status: Enrolled
Student number: 202511693
Name: Charles Gabriel Marcelo
Phone: 67676776767
Zip code: 67676
Classification (or N/A): Regular
College: COMPUTER STUDIES
Program: BSCSSE
Year level: 2nd
Term: 1
School year: 2026-2027
Address: 123 Test. St
```

```
---------- SUBJECT UNIT ENTRY ----------
Enter whole units (0-30); 0 means not enrolled.
Tuition: PHP 1000/unit. Laboratory: PHP 2500 per enrolled lab subject.

1. CCS0021  - INFORMATION MANAGEMENT (LEC)
Number of units: 2

2. CCS0021L - INFORMATION MANAGEMENT (LAB)
Number of units: 1

3. CS0001   - DISCRETE STRUCTURES 1
Number of units: 3

4. CS0003   - COMPUTER SYSTEMS & ARCHITECTURE
Number of units: 2

5. CS0003L  - COMPUTER SYSTEMS & ARCHITECTURE (LAB)
Number of units: 1

6. CS0070   - OBJECT ORIENTED PROGRAMMING
Number of units: 2

7. CS0070L  - OBJECT ORIENTED PROGRAMMING-LAB
Number of units: 1

8. GED0021  - SPECIALIZED ENGLISH PROGRAM 2
Number of units: 3

9. GED0055  - MATHEMATICAL ANALYSIS
9. GED0055  - MATHEMATICAL ANALYSIS
Number of units: 3

10. GED0081  - COLLEGE PHYSICS 1 LECTURE
Number of units: 2

11. GED0081L - COLLEGE PHYSICS 1 LABORATORY
Number of units: 1

---------- ENROLLMENT SUMMARY ----------
CCS0021  - INFORMATION MANAGEMENT (LEC) | Units: 2
CCS0021L - INFORMATION MANAGEMENT (LAB) | Units: 1
CS0001   - DISCRETE STRUCTURES 1 | Units: 3
CS0003   - COMPUTER SYSTEMS & ARCHITECTURE | Units: 2
CS0003L  - COMPUTER SYSTEMS & ARCHITECTURE (LAB) | Units: 1
CS0070   - OBJECT ORIENTED PROGRAMMING | Units: 2
CS0070L  - OBJECT ORIENTED PROGRAMMING-LAB | Units: 1
GED0021  - SPECIALIZED ENGLISH PROGRAM 2 | Units: 3
GED0055  - MATHEMATICAL ANALYSIS | Units: 3
GED0081  - COLLEGE PHYSICS 1 LECTURE | Units: 2
GED0081L - COLLEGE PHYSICS 1 LABORATORY | Units: 1

---------- TUITION ASSESSMENT ----------
Total units: 21
Enrolled laboratory subjects: 4
Tuition (units x PHP 1000): PHP 21000.00
Laboratory fees (lab subjects x PHP 2500): PHP 10000.00
TOTAL AMOUNT DUE: PHP 31000.00
@kizern →/workspaces/bananabread/FLAB3 (main) $
```

SOURCE CODE:

```
global _start
```

```
%define SYS_EXIT       1
%define SYS_READ       3
%define SYS_WRITE      4
%define FIELD_SIZE     96
%define UNIT_PRICE     1000
%define LAB_PRICE      2500
%define MAX_UNITS      30

%macro PRINT 1
    push eax
    mov eax, %1
    call print_string
    pop eax
%endmacro

%macro PRINT_NUMBER 1
    push eax
    mov eax, %1
    call print_uint
    pop eax
%endmacro

%macro ASK_TEXT 3
    ; Arguments: prompt address, buffer address, buffer capacity.
    push ebx
    push ecx
    push edx
    mov ebx, %1
    mov ecx, %2
    mov edx, %3
    call ask_text
    pop edx
    pop ecx
    pop ebx
%endmacro

SECTION .data
    banner db 10, '========== SOLAR ENROLLMENT ==========', 10, 0
    info_heading db 10, '---------- STUDENT INFORMATION ----------', 10, 0
    subject_heading db 10, '---------- SUBJECT UNIT ENTRY ----------', 10
                db 'Enter whole units (0-30); 0 means not enrolled.', 10
```

```
                db 'Tuition: PHP 1000/unit. Laboratory: PHP 2500 per enrolled lab subject.',
10, 0
    assessment_heading db 10, '---------- ENROLLMENT SUMMARY ----------', 10, 0
    fee_heading db 10, '---------- TUITION ASSESSMENT ----------', 10, 0
    newline db 10, 0
    separator db '. ', 0
    units_suffix db ' | Units: ', 0
    unit_prompt db 'Number of units: ', 0
    invalid_text db 'Please enter 1-95 characters.', 10, 0
    invalid_units db 'Invalid units. Enter a whole number from 0 to 30.', 10, 0
    input_stopped db 10, 'Input ended or could not be read. Assessment cancelled.', 10, 0
    total_units_label db 'Total units: ', 0
    lab_count_label db 'Enrolled laboratory subjects: ', 0
    tuition_label db 'Tuition (units x PHP 1000): PHP ', 0
    lab_fee_label db 'Laboratory fees (lab subjects x PHP 2500): PHP ', 0
    total_label db 'TOTAL AMOUNT DUE: PHP ', 0
    cents db '.00', 10, 0

    p_status db 'Enrollment status: ', 0
    p_student db 'Student number: ', 0
    p_name db 'Name: ', 0
    p_phone db 'Phone: ', 0
    p_zip db 'Zip code: ', 0
    p_class db 'Classification (or N/A): ', 0
    p_college db 'College: ', 0
    p_program db 'Program: ', 0
    p_year db 'Year level: ', 0
    p_term db 'Term: ', 0
    p_school_year db 'School year: ', 0
    p_address db 'Address: ', 0

    field_prompts dd p_status, p_student, p_name, p_phone, p_zip, p_class
                  dd p_college, p_program, p_year, p_term, p_school_year, p_address
    FIELD_COUNT equ ($ - field_prompts) / 4

    s1 db 'CCS0021  - INFORMATION MANAGEMENT (LEC)', 0
    s2 db 'CCS0021L - INFORMATION MANAGEMENT (LAB)', 0
    s3 db 'CS0001   - DISCRETE STRUCTURES 1', 0
    s4 db 'CS0003   - COMPUTER SYSTEMS & ARCHITECTURE', 0
    s5 db 'CS0003L  - COMPUTER SYSTEMS & ARCHITECTURE (LAB)', 0
    s6 db 'CS0070   - OBJECT ORIENTED PROGRAMMING', 0
    s7 db 'CS0070L  - OBJECT ORIENTED PROGRAMMING-LAB', 0
    s8 db 'GED0021  - SPECIALIZED ENGLISH PROGRAM 2', 0
```

```
    s9 db 'GED0055  - MATHEMATICAL ANALYSIS', 0
    s10 db 'GED0081  - COLLEGE PHYSICS 1 LECTURE', 0
    s11 db 'GED0081L - COLLEGE PHYSICS 1 LABORATORY', 0

    subjects dd s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11
    SUBJECT_COUNT equ ($ - subjects) / 4
    is_lab db 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1

SECTION .bss
    student_info resb FIELD_COUNT * FIELD_SIZE
    subject_units resd SUBJECT_COUNT
    input_buffer resb FIELD_SIZE
    input_char resb 1
    number_buffer resb 11
    total_units resd 1
    lab_count resd 1
    tuition_fee resd 1
    laboratory_fee resd 1
    total_fee resd 1

SECTION .text
_start:
    PRINT banner
    call get_student_info
    call display_student_info
    call get_subject_units
    call calculate_fees
    call display_assessment
    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80

get_student_info:
    pushad
    xor esi, esi
.next:
    imul edi, esi, FIELD_SIZE
    add edi, student_info
    ASK_TEXT [field_prompts + esi*4], edi, FIELD_SIZE
    inc esi
    cmp esi, FIELD_COUNT
    jb .next
    popad
```

```
    ret

display_student_info:
    pushad
    PRINT info_heading
    xor esi, esi
.next:
    PRINT [field_prompts + esi*4]
    imul edi, esi, FIELD_SIZE
    add edi, student_info
    PRINT edi
    PRINT newline
    inc esi
    cmp esi, FIELD_COUNT
    jb .next
    popad
    ret

get_subject_units:
    pushad
    PRINT subject_heading
    xor esi, esi
.next:
    PRINT newline
    lea eax, [esi + 1]
    PRINT_NUMBER eax
    PRINT separator
    PRINT [subjects + esi*4]
    PRINT newline
    call read_units
    mov [subject_units + esi*4], eax
    inc esi
    cmp esi, SUBJECT_COUNT
    jb .next
    popad
    ret

read_units:
    push ebx
    push ecx
    push edx
    push esi
.retry:
```

```
    ASK_TEXT unit_prompt, input_buffer, FIELD_SIZE
    mov esi, input_buffer
    xor eax, eax
.digit:
    movzx edx, byte [esi]
    test edx, edx
    jz .done
    cmp edx, '0'
    jb .invalid
    cmp edx, '9'
    ja .invalid
    sub edx, '0'
    imul eax, eax, 10
    add eax, edx
    cmp eax, MAX_UNITS
    ja .invalid
    inc esi
    jmp .digit
.invalid:
    PRINT invalid_units
    jmp .retry
.done:
    pop esi
    pop edx
    pop ecx
    pop ebx
    ret

calculate_fees:
    pushad
    mov dword [total_units], 0
    mov dword [lab_count], 0
    xor esi, esi
.next:
    mov eax, [subject_units + esi*4]
    add [total_units], eax
    test eax, eax
    jz .skip_lab
    cmp byte [is_lab + esi], 1
    jne .skip_lab
    inc dword [lab_count]
.skip_lab:
    inc esi
```

```
    cmp esi, SUBJECT_COUNT
    jb .next
    mov eax, [total_units]
    imul eax, eax, UNIT_PRICE
    mov [tuition_fee], eax
    mov edx, [lab_count]
    imul edx, edx, LAB_PRICE
    mov [laboratory_fee], edx
    add eax, edx
    mov [total_fee], eax
    popad
    ret

display_assessment:
    pushad
    PRINT assessment_heading
    xor esi, esi
.next:
    PRINT [subjects + esi*4]
    PRINT units_suffix
    PRINT_NUMBER [subject_units + esi*4]
    PRINT newline
    inc esi
    cmp esi, SUBJECT_COUNT
    jb .next
    PRINT fee_heading
    PRINT total_units_label
    PRINT_NUMBER [total_units]
    PRINT newline
    PRINT lab_count_label
    PRINT_NUMBER [lab_count]
    PRINT newline
    PRINT tuition_label
    PRINT_NUMBER [tuition_fee]
    PRINT cents
    PRINT lab_fee_label
    PRINT_NUMBER [laboratory_fee]
    PRINT cents
    PRINT total_label
    PRINT_NUMBER [total_fee]
    PRINT cents
    popad
    ret
```

```
ask_text:
    push ebx
    push ecx
    push edx
.retry:
    PRINT ebx
    call read_line
    cmp eax, -1
    je input_error
    cmp eax, 0
    jle .invalid
    pop edx
    pop ecx
    pop ebx
    ret
.invalid:
    PRINT invalid_text
    jmp .retry

read_line:
    push ebx
    push ecx
    push edx
    push esi
    push edi
    push ebp
    mov edi, ecx
    mov ebp, edx
    dec ebp
    xor esi, esi
.read:
    mov eax, SYS_READ
    xor ebx, ebx
    mov ecx, input_char
    mov edx, 1
    int 0x80
    cmp eax, -4
    je .read
    test eax, eax
    js .error
    jz .eof
    mov al, [input_char]
```

```
    cmp al, 10
    je .finished
    cmp al, 13
    je .read
    cmp esi, ebp
    jae .overflow
    mov [edi + esi], al
    inc esi
    jmp .read
.overflow:
    lea esi, [ebp + 1]
    jmp .read
.eof:
    test esi, esi
    jz .error
.finished:
    cmp esi, ebp
    ja .too_long
    mov byte [edi + esi], 0
    mov eax, esi
    jmp .return
.too_long:
    mov byte [edi + ebp], 0
    mov eax, -2
    jmp .return
.error:
    mov eax, -1
.return:
    pop ebp
    pop edi
    pop esi
    pop edx
    pop ecx
    pop ebx
    ret

print_string:
    pushad
    mov ecx, eax
    xor edx, edx
.length:
    cmp byte [ecx + edx], 0
    je .write
```

```
    inc edx
    jmp .length
.write:
    test edx, edx
    jz .done
    mov eax, SYS_WRITE
    mov ebx, 1
    int 0x80
    cmp eax, -4
    je .write
    test eax, eax
    jle .failed
    add ecx, eax
    sub edx, eax
    jmp .write
.done:
    popad
    ret
.failed:
    mov eax, SYS_EXIT
    mov ebx, 1
    int 0x80

print_uint:
    pushad
    mov edi, number_buffer + 10
    mov byte [edi], 0
    mov ebx, 10
.convert:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    test eax, eax
    jnz .convert
    mov eax, edi
    call print_string
    popad
    ret

input_error:
    PRINT input_stopped
```

```
mov eax, SYS_EXIT
mov ebx, 1
int 0x80
```